

Storycards

Product Requirements & Technical Specification

Web Application — v1.0

A digital version of the screenwriter's 3x5 index card method. Drag-and-drop story beats, instant AI summaries, shuffle-to-discover, and shareable URLs — built for writers who think in fragments.

1. Product Overview

1.1 Problem

The analog index card method is one of the most widely used story development tools in professional screenwriting. Writers like Vince Gilligan, Aaron Sorkin, and John August have credited it as core to their process. The method works because it externalizes the story, makes structure physical and movable, and allows non-linear thinking.

The problem: ideas die between the card and the computer. Writers either forget to digitize, lose the physical cards, or can't maintain organization across a large project. There is no digital product that faithfully replicates the physical experience while adding what software can uniquely offer.

1.2 Solution

Storycards is a browser-based story development tool that mirrors the physical index card experience while adding: persistent cloud storage, AI-powered narrative summarization, shareable URLs, and organizational layers (color, category, character) that would be impractical on physical cards.

1.3 Target Users

- Amateur and professional screenwriters developing feature films, TV pilots, and short films
- Novelists and narrative game designers who use beat-sheet methods
- Writers in writers' rooms or creative partnerships needing a shared, real-time board
- Students in film programs learning story structure

1.4 Core Principles

- Speed first: a card should be creatable in under 10 seconds
 - Tactile feel: drag, drop, shuffle, and reorder must feel physical and responsive
 - Non-linear thinking: the tool should reward chaotic input and make structure emergent, not imposed
 - Minimal friction: no account required to start; save and share should be one click
-

2. Feature Requirements

2.1 Card Creation & Editing

Each card represents a single story beat, scene, plot point, character note, thematic idea, or world-building element.

Required fields per card:

- Title (required) — short headline for the beat, max 80 chars
- Body text (optional) — the beat description, max 500 chars
- Category (required) — one of: Plot, Character, World Building, Theme, Twist, Scene
- Characters (optional) — comma-separated list of character names tagged to this card
- Position — numeric order within the deck, auto-assigned, manually overridable

Behaviors:

- New card form opens in a side panel without leaving the board
- Clicking Edit on any card opens the same panel pre-filled
- Saving a card immediately updates the board without full page reload
- Cards can be deleted with a confirmation prompt
- Keyboard shortcut N opens new card form

2.2 Board & Card Display

The main board displays all cards as a responsive grid of index cards. Visual design should evoke physical stationery: cream backgrounds, typeface that feels hand-chosen, subtle paper texture.

Each card displays:

- Category color band at the top (unique color per category, consistent across the app)
- Category label and position number
- Title in display typeface
- Body text in smaller weight
- Character tags as inline pills
- Edit and Delete buttons in the card footer
- Drag handle

Board layout:

- Auto-fill grid, minimum card width 220px, gap 14px
- Cards scroll vertically; board header stays fixed

- Empty state with prompt to add first card when deck is empty

2.3 Drag-and-Drop Reordering

This is the central interaction. It must feel fast, accurate, and satisfying.

- Cards are draggable by the drag handle or the full card body
- Dragging a card over another card shows a drop target indicator
- On drop, the card takes the position of the target; all other cards shift accordingly
- Position numbers update immediately on the board
- Drag state: dragged card becomes semi-transparent, slight rotation
- On mobile (touch): touch-drag must also work via touch events or a touch-drag library
- Undo last reorder with Cmd/Ctrl+Z

2.4 Shuffle

The shuffle feature is a core discovery tool. Writers should be able to randomize their deck and read the AI summary of the new order to discover unexpected narrative structures.

- Shuffle button randomizes all visible cards' positions
- Animation: cards briefly scatter before settling in new order
- After shuffling, the Story Order sidebar and AI Summary reflect the new sequence
- Shuffle respects the active category filter (only shuffles visible cards within filtered view)
- Shuffle history: last 5 shuffle states are recoverable via Undo

2.5 Filtering & Organization

Writers need to isolate one thread of their story without destroying the overall deck.

- Filter bar with toggle pills for each category: All, Plot, Character, World Building, Theme, Twist, Scene
- Filtering shows only matching cards; position numbers reflect the full deck order (not the filtered subset)
- Filter state persists across browser sessions via localStorage
- Character filter: a secondary filter dropdown shows all unique character names extracted from card tags; selecting one shows only cards where that character is tagged
- Search: text input in the top bar that live-filters cards by title and body text

2.6 AI Story Summarizer

The summarizer reads the current deck in its current order and returns a short narrative overview. This is the key feature that makes digital superior to physical — the story can 'speak back' to you.

- Triggered by "Summarize" button in the toolbar
- Sends all cards (in current order) to Claude API via the app's own backend
- Prompt instructs Claude to write a 3–5 sentence atmospheric story overview, writer-to-writer in tone
- Prompt explicitly asks Claude to summarize the order AS IS — not the 'correct' order — and to note if non-linear ordering creates interesting effects

- Summary displays in the right sidebar panel
- Re-summarize button appears after first generation; re-runs if card order has changed
- Summary is cached by deck state hash; does not re-run if nothing has changed
- Character index below the summary: all unique character names with card count

API call behavior:

- Calls app backend endpoint, not client-side directly (API key stays server-side)
- Loading state with animated dots while waiting
- Error state if API call fails, with retry button

2.7 Save & Share

This is what separates the app from a throwaway tool. Every deck must be shareable and recoverable.

- First save creates a unique deck ID and URL (e.g., `storycards.app/deck/abc123`)
- URL is copyable from a share button in the toolbar
- Deck title field in the toolbar; defaults to “Untitled deck”
- Auto-save: deck saves to backend every 30 seconds if changes exist, and on every explicit card action (add, edit, delete, reorder)
- Save indicator in toolbar: “Saved” / “Saving...” / “Unsaved changes”
- Loading a shared URL shows the deck in read-only mode by default
- Edit mode: clicking “Edit this deck” in a shared view creates a new fork with a new URL
- No account required for MVP; ownership is URL-based (whoever has the URL can edit)
- Export: download deck as JSON file for offline backup

2.8 Story Order View

A compact linear view of the current deck sequence for at-a-glance structure review.

- Accessible via sidebar tab
- Shows all cards in position order: number, color dot, title, category label
- Clicking a card in the list scrolls to and highlights that card on the board
- Shuffle button available from within this view

3. Technical Architecture

3.1 Recommended Stack

This stack is chosen for fast deployment, minimal ops overhead, and suitability for a solo or small-team build. Everything runs on free tiers until meaningful scale.

Layer	Choice
-------	--------

Frontend framework	React 18 + Vite
Styling	Tailwind CSS + CSS variables for theming
Drag and drop	dnd-kit (accessible, touch-compatible, performant)
State management	Zustand (lightweight, no boilerplate)
Backend	Next.js API routes (or Express if deployed separately)
Database	Supabase (Postgres + realtime + free tier)
AI API	Anthropic Claude API (claude-sonnet-4-20250514)
Hosting	Vercel (frontend + API routes)
Auth (optional v2)	Supabase Auth

3.2 Database Schema

decks table

Column	Type / Notes
id	uuid, primary key, auto-generated
title	text, default 'Untitled deck'
created_at	timestamp
updated_at	timestamp
owner_token	text — random string stored in user's localStorage, used for edit authorization

cards table

Column	Type / Notes
id	uuid, primary key
deck_id	uuid, foreign key → decks.id
title	text, required
body	text
category	enum: plot character world theme twist scene
characters	text[] (array of character name strings)
position	integer (1-indexed, unique per deck)
created_at	timestamp
updated_at	timestamp

3.3 API Routes

Route	Description
POST /api/decks	Create new deck, returns deck ID and owner token
GET /api/decks/:id	Fetch deck + all cards (sorted by position)
PATCH /api/decks/:id	Update deck title (requires owner token in header)
POST /api/decks/:id/cards	Add card to deck
PATCH /api/cards/:id	Update card (title, body, category, chars, position)
DELETE /api/cards/:id	Delete card
POST /api/decks/:id/reorder	Batch update positions after drag/drop or shuffle
POST /api/decks/:id/summarize	Trigger AI summary — proxies to Anthropic API

3.4 Frontend File Structure

src/	
components/	
Card.jsx	– individual index card
Board.jsx	– card grid with dnd-kit provider
Sidebar.jsx	– add/edit form, story order, summary panel
TopBar.jsx	– title, actions, filter pills
SummaryPanel.jsx	– AI summary display
OrderList.jsx	– linear narrative order view
hooks/	
useDeck.js	– deck fetch, save, auto-save logic
useCards.js	– card CRUD operations
useSummary.js	– AI summary fetch, cache
store/	
deckStore.js	– Zustand store (cards, deck, UI state)
lib/	
api.js	– all fetch calls to /api/*
constants.js	– category definitions, colors, labels
pages/	
index.jsx	– redirect to new deck
deck/[id].jsx	– main app page

3.5 AI Summarizer Prompt

Send this exact prompt structure to the Anthropic API. The model should be claude-sonnet-4-20250514.

System: You are a story development assistant working with a screenwriter. Be concise and evocative. Write like a story editor talking to a writer, not like a book report.

User prompt (constructed dynamically):

Below are index cards from a screenplay in their current order.
Write a brief story summary (3-5 sentences) that captures the narrative arc, tone, and emotional core AS THE CARDS ARE CURRENTLY ORDERED – not what you think the 'correct' order should be. If the order creates interesting non-linear effects or juxtapositions, note that. No bullet points. Be direct and atmospheric. Writer-to-writer tone.

```
Cards:
[Card 1] [Plot] "Opening image": We meet Elena walking the dead orchard...
[Card 2] [Twist] "The land's secret": Hidden beneath the oldest tree...
...
```

Max tokens: 400. Temperature: 0.7.

4. UX & Design Specification

4.1 Design Direction

The visual language is editorial stationery meets film production. Think: a real index card pinned to a corkboard, but clean. Cream backgrounds, ink-colored text, typefaces that feel chosen not defaulted to, and a color system that feels like a set of colored pens rather than a software UI.

Avoid: gradients, drop shadows, purple-on-white, rounded blobs, and anything that screams "SaaS dashboard".

4.2 Typography

Use	Font
Card titles, deck title	DM Serif Display (Google Fonts)
Body text, UI labels	Figtree (Google Fonts)
Card position numbers	DM Mono
Code / data display	DM Mono

4.3 Color System

Background: #FAF8F3 (warm cream). Ink: #1A1714. Muted: #6B665E. Faint: #B8B3AA.

Category	Color (hex) / Usage
Plot	#C8430A — burnt orange
Character	#1A6B5A — forest green
World Building	#2952A3 — deep blue
Theme	#7B3FA8 — purple
Twist	#B8830A — amber
Scene	#555555 — neutral gray

4.4 Key Interaction States

- Card hover: subtle lift in shadow
- Card dragging: card goes semi-transparent + slight rotation, cursor becomes grabbing
- Drop target: dashed outline in plot orange on the target card

- Saving: toolbar shows “Saving...” with spinner, then “Saved” on success
- Shuffle: 200ms scatter animation before cards settle
- Form validation: title field shows red border + message if empty on submit

4.5 Responsive Behavior

- Desktop ($\geq 1200\text{px}$): sidebar open by default at 340px, board fills remaining width
- Tablet (768–1199px): sidebar collapsed by default, toggleable via hamburger button
- Mobile ($< 768\text{px}$): sidebar becomes bottom sheet; board shows single-column card list
- Touch drag-and-drop required on mobile (dnd-kit supports this natively)

5. Build Sequence

Follow this order to maintain a working app at each stage. Don’t build the AI features before the core card CRUD is solid.

Phase 1: Core (Week 1)

1. Scaffold project: Vite + React + Tailwind + Zustand + dnd-kit
2. Build Card component with all display states
3. Build Board with responsive grid and dnd-kit drag-and-drop
4. Build Sidebar with new card form
5. Wire up Zustand store: add, edit, delete, reorder cards in local state
6. Verify drag-and-drop reorder works correctly end-to-end

Phase 2: Persistence (Week 2)

7. Set up Supabase project, create decks and cards tables
8. Build Next.js API routes for deck and card CRUD
9. Connect frontend to API: create deck on first load, save cards on every action
10. Implement auto-save with debounce (30s or on action)
11. Add shareable URL: deck loads from URL on visit
12. Add owner token logic: edit mode vs. read-only mode

Phase 3: AI + Polish (Week 3)

13. Build `/api/decks/:id/summarize` route with Anthropic API call
14. Build SummaryPanel component with loading, error, and success states
15. Wire up filter pills with category and character filtering
16. Build Story Order sidebar tab
17. Add shuffle animation
18. Add keyboard shortcuts (N for new card, Cmd+Z for undo)
19. Add JSON export
20. Mobile touch drag testing and fixes

Phase 4: Launch (Week 4)

21. Deploy to Vercel, connect Supabase environment variables
22. Add og:image and meta tags for link sharing
23. Test on Chrome, Safari, Firefox, iOS Safari, Android Chrome
24. Write a one-paragraph onboarding overlay for first-time visitors
25. Soft launch

6. Environment Variables

Variable	Description
ANTHROPIC_API_KEY	Your Anthropic API key — server-side only, never expose to client
SUPABASE_URL	Your Supabase project URL
SUPABASE_ANON_KEY	Supabase anon public key (safe for client)
SUPABASE_SERVICE_KEY	Supabase service role key — server-side only
NEXT_PUBLIC_APP_URL	Base URL of the deployed app (for share links)

7. Prompt for Claude in Terminal

Use this as your initial prompt when starting the build in Claude Code (terminal) or VS Code with Claude. Paste it as-is.

Build a web application called Storycards using React 18 + Vite + Tailwind CSS + Zustand for state + dnd-kit for drag-and-drop. The app is a digital version of the screenwriter 3x5 index card method. Start with Phase 1 only: scaffold the project, build the Card component, Board with dnd-kit drag-and-drop reorder, and Sidebar with new card form. Wire everything to a Zustand store (no backend yet — local state only). Design spec: - Background: #FAF8F3. Text: #1A1714. Fonts: DM Serif Display (card titles), Figtree (UI), DM Mono (position numbers). - Category colors: Plot #C8430A, Character #1A6B5A, World #2952A3, Theme #7B3FA8, Twist #B8830A, Scene #555 - Cards show: category color band, category label, position number, title, body text, character tags, edit/delete buttons, drag handle - Board: auto-fill grid, min card width 220px, gap 14px - Sidebar (340px, right side): Add/Edit card form with fields: category selector, title, body, character tags (tag input), position selector - Top bar: deck title, + New Card button, Shuffle button, Summarize button, category filter pills Drag-and-drop: use dnd-kit. Dragging a card over another swaps their positions; all other positions shift accordingly. Dragged card gets opacity 0.4 + 2deg rotation. Load 4–6 sample cards so the board is populated on first render. Follow the file structure in the spec doc.

After Phase 1 is working, use this Phase 2 prompt:

Phase 1 is complete. Now add persistence using Supabase and Next.js API routes. Schema: decks table (id uuid, title text, created_at, updated_at, owner_token text). cards table (id uuid, deck_id uuid FK, title text, body text, category enum, characters text[], position integer, created_at, updated_at). API routes: POST /api/decks, GET /api/decks/:id, PATCH /api/decks/:id, POST /api/decks/:id/cards, PATCH /api/cards/:id, DELETE /api/cards/:id, POST /api/decks/:id/reorder. Behavior: on first load with no deck ID in URL, create a new deck via POST /api/decks and redirect to /deck/[id]. Store owner_token in localStorage. Load deck from URL on visit. Auto-save on every card action and every 30 seconds. Show save state in toolbar. Shared URL opens in read-only mode unless localStorage has matching owner_token.

8. Future Features (v2+)

- User accounts and deck library
- Real-time multiplayer (writers' room mode) via Supabase realtime
- Act structure overlay (Act 1 / Act 2A / Midpoint / Act 2B / Act 3 dividers on the board)
- Card connections: draw a line between two cards to show causal or thematic relationship
- Beat sheet templates: pre-load Save the Cat, Hero's Journey, or Dan Harmon Story Circle as card skeletons
- Voice-to-card: dictate a beat on mobile, AI parses it into a formatted card
- PDF export: print deck as formatted beat sheet
- Version history: restore any previous deck state